

# Product Requirement Document (PRD) dan Spesifikasi Arsitektur Sistem: Serverless Client-Side Photo Proofing Platform Berbasis Google Drive, shadcn/ui, dan GitHub Pages PWA

## Evaluasi Lanskap Industri dan Analisis Kebutuhan Sistem

Industri fotografi komersial, pernikahan, dan dokumentasi korporat bertumpu pada efisiensi siklus peninjauan visual antara fotografer dan klien. Fase *photo proofing*, yakni proses kurasi di mana klien memilih foto mentah (*preview*) untuk diproses ke tahap penyuntingan akhir (*color grading* dan *retouching*), sering kali menjadi titik penghambat operasional terbesar. Sebagian besar pelaku industri mengunggah ratusan hingga ribuan berkas ke dalam penyimpanan cloud seperti Google Drive, kemudian meminta klien mencatat nomor seri berkas atau mengambil tangkapan layar secara manual melalui aplikasi perpesanan WhatsApp. Pendekatan konvensional ini menciptakan friksi komunikasi yang tinggi, memicu ketidaksesuaian pencatatan nama berkas (*human error*), serta memperpanjang durasi pasca-produksi secara signifikan. Layanan Software-as-a-Service (SaaS) komersial khusus industri fotografi, seperti Pixieset, ShootProof, atau CloudSpot, menyediakan antarmuka seleksi yang terstruktur. Namun, platform-platform tersebut memberlakukan biaya langganan bulanan yang relatif tinggi serta memaksa fotografer menduplikasi kapasitas penyimpanan di luar langganan Google Workspace atau Google One yang telah mereka miliki. Kebutuhan industri mengarah pada pembentukan sistem mandiri (*self-hosted client-side platform*) yang mampu mengubah direktori Google Drive publik fotografer menjadi galeri web interaktif yang modern, responsif, dan aman. Pilihan infrastruktur berbasis GitHub Pages menawarkan keunggulan *zero infrastructure cost* dan reliabilitas *uptime* tinggi karena seluruh aset didistribusikan melalui Content Delivery Network (CDN) global GitHub. Tantangan rekayasa mendasar dari arsitektur ini adalah ketiadaan *server-side backend* konvensional untuk mengelola sesi, memproses token rahasia, atau menyimpan data transaksi. Dokumen ini menetapkan Product Requirement Document (PRD) dan cetak biru arsitektur lengkap bagi platform *photo proofing* berbasis React, TypeScript, Tailwind CSS, dan komponen antarmuka shadcn/ui. Solusi ini mengintegrasikan fungsionalitas Progressive Web App (PWA) untuk instalasi perangkat seluler dan kapabilitas luring (*offline-first*), mekanisme Content Security Policy (CSP) statis, manajemen sesi lokal terisolasi, serta automasi penuh pipeline CI/CD GitHub Actions.

## Arsitektur Integrasi Google Drive API dan Ekstraksi Konten

Penyusunan aplikasi *photo proofing* web statis yang mengandalkan Google Drive menghadapi restriksi teknis terkait kebijakan keamanan Google Workspace. Pada awal tahun 2024, Google secara bertahap menonaktifkan URL sematan langsung berkas publik berbasis endpoint lama seperti [https://drive.google.com/uc?export=view&id=FILE\\_ID](https://drive.google.com/uc?export=view&id=FILE_ID), yang menghasilkan respons kegagalan 403 Forbidden pada elemen HTML `<img>` peramban pihak ketiga. Rekayasa integrasi modern harus mengombinasikan ekstraksi metadata melalui REST API dengan rendering visual via edge CDN Google User Content.

Alur ekstraksi konten dimulai dari fotografer yang menyediakan tautan folder Google Drive yang telah dikonfigurasi dengan izin akses publik (*Anyone with the link can view*). Klien atau aplikasi kemudian mengekstrak pengenalan unik (*Folder ID*) dari parameter URL. Aplikasi peramban mengeksekusi kueri HTTP GET ke endpoint Google Drive API v3:

[https://www.googleapis.com/drive/v3/files?q='{FOLDER\\_ID}'+in+parents+and+trashed=false&fields=files\(id,name,mimeType,thumbnailLink,imageMediaMetadata,size\)&pageSize=1000&key={API\\_KEY}](https://www.googleapis.com/drive/v3/files?q='{FOLDER_ID}'+in+parents+and+trashed=false&fields=files(id,name,mimeType,thumbnailLink,imageMediaMetadata,size)&pageSize=1000&key={API_KEY})

Metadata yang diterima dinormalisasi oleh layer data aplikasi ke dalam format JSON lokal.

Untuk proses rendering gambar pada antarmuka pengguna tanpa memicu pemblokiran CORS atau pelanggaran otorisasi, aplikasi menyusun tautan langsung menggunakan CDN

[https://lh3.googleusercontent.com/d/{FILE\\_ID}=w{WIDTH}](https://lh3.googleusercontent.com/d/{FILE_ID}=w{WIDTH}) atau endpoint

[https://drive.google.com/thumbnail?id={FILE\\_ID}&sz=w{WIDTH}](https://drive.google.com/thumbnail?id={FILE_ID}&sz=w{WIDTH}). Parameter lebar dinamis (w400 untuk *grid* pratinjau dan w1600 untuk peninjauan *modal*) menghemat konsumsi bandwidth jaringan dan mempercepat proses rendering pada perangkat bergerak.

| Dimensi Parameter           | Google Drive API v3 (files.list)           | Google Apps Script (GAS) Web App                    | Direct CDN Formatting (lh3.googleusercontent)  |
|-----------------------------|--------------------------------------------|-----------------------------------------------------|------------------------------------------------|
| <b>Prasyarat Hak Akses</b>  | Folder publik ( <i>Anyone with link</i> )  | Akses internal akun fotografer                      | Folder/File publik ( <i>Anyone with link</i> ) |
| <b>Kebutuhan Kredensial</b> | API Key dengan restriksi HTTP Referrer     | Tokenless bagi klien ( <i>Run as script owner</i> ) | Tanpa kredensial ( <i>Direct asset link</i> )  |
| <b>Latensi Eksekusi</b>     | 150 – 300 ms (REST API standar)            | 800 – 2500 ms (Serverless engine overhead)          | 50 – 150 ms (Google Edge CDN)                  |
| <b>Batasan Operasional</b>  | Kuota kueri Google Cloud (20.000 req/100s) | Batas harian eksekusi komputasi GAS                 | Proteksi IP rate-limiting Google               |
| <b>Peran Komponen</b>       | Enumerasi struktur berkas dan metadata     | Pipeline penulisan data dan transaksi formulir      | Rendering visual elemen media antarmuka        |

Dalam lingkungan statis GitHub Pages, rahasia klien (*client secret*) tidak boleh ditanamkan ke dalam repositori atau dibundel ke dalam kompilasi JavaScript. Sistem secara ketat membedakan antara API Key publik dan token privat. API Key yang digunakan dalam aplikasi dibatasi melalui Google Cloud Console menggunakan dua lapis pertahanan: pembatasan akses hanya untuk Google Drive API, serta pengikatan domain via HTTP Referrer Restrictions ke alamat produksi GitHub Pages dan host pengembangan lokal.

Penanganan CORS pada peramban sering kali gagal apabila pengembang mencampuradukkan parameter autentikasi, seperti menyertakan API Key bersamaan dengan header Authorization: Bearer <token> pada permintaan baca. Arsitektur ini menetapkan kebijakan pemanggilan REST API v3 murni menggunakan kueri API Key untuk direktori publik, sehingga menghindari kegagalan *preflight* OPTIONS dan menjaga kestabilan komunikasi data

langsung dari peramban klien.

## Model Keamanan Siber dan Proteksi Data pada Infrastruktur Tanpa Server

Infrastruktur GitHub Pages meniadakan lapisan pelindung *backend* konvensional seperti Web Application Firewall (WAF) terkelola, proksi terbalik (*reverse proxy*), atau manipulasi header HTTP dinamis. Oleh sebab itu, seluruh postur pertahanan keamanan aplikasi harus dirancang secara mandiri di sisi klien dengan memanfaatkan kapabilitas peramban modern.

### Penegakan Content Security Policy (CSP) Deklaratif

Mengingat GitHub Pages tidak mengizinkan penambahan header respons HTTP khusus melalui berkas konfigurasi server, penegakan integritas eksekusi skrip dan pemuatan aset harus dilakukan melalui tag `<meta http-equiv="Content-Security-Policy">` pada elemen `<head>` berkas `index.html`. Kebijakan keamanan dirancang menggunakan prinsip hak akses minimum (*least privilege*) untuk mencegah serangan Cross-Site Scripting (XSS) dan pembajakan sesi data:

```
<meta http-equiv="Content-Security-Policy" content="
  default-src 'self';
  script-src 'self' https://accounts.google.com
https://apis.google.com;
  style-src 'self' 'unsafe-inline' https://fonts.googleapis.com;
  font-src 'self' https://fonts.gstatic.com;
  img-src 'self' data: blob: https://lh3.googleusercontent.com
https://*.googleusercontent.com https://drive.google.com;
  connect-src 'self' https://www.googleapis.com
https://script.google.com https://script.googleusercontent.com;
  frame-src https://accounts.google.com;
  object-src 'none';
  base-uri 'self';
  form-action 'none';
">
```

Direktif `object-src 'none'` menonaktifkan eksekusi plugin peramban lawas, sedangkan restriksi pada `connect-src` menjamin bahwa aplikasi peramban hanya diizinkan mengirimkan transmisi XHR/Fetch ke endpoint resmi Google API dan webhook Google Apps Script.

| Kategori Ancaman    | Tingkat Keparahan | Vektor Kerentanan                     | Rekayasa Mitigasi Sistem                                                           |
|---------------------|-------------------|---------------------------------------|------------------------------------------------------------------------------------|
| Eksfiltrasi API Key | Menengah          | Pembongkaran bundle JavaScript publik | Pembatasan HTTP Referrer dan isolasi skop API eksklusif pada Google Cloud Console. |
| Injeksi Stored XSS  | Tinggi            | Penanaman payload jahat pada metadata | Sanitasi wajib seluruh string dari respons API                                     |

| Kategori Ancaman                    | Tingkat Keparahan | Vektor Kerentanan                                | Rekayasa Mitigasi Sistem                                                                                  |
|-------------------------------------|-------------------|--------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
|                                     |                   | nama file foto                                   | menggunakan pustaka DOMPurify sebelum injeksi DOM.                                                        |
| <b>Intersepsi Data Klien</b>        | Kritis            | Penyadapan daftar seleksi foto oleh pihak ketiga | Isolasi data pada penyimpanan lokal IndexedDB peramban klien tanpa pembagian sesi terpusat.               |
| <b>Kebocoran Webhook</b>            | Tinggi            | Pengiriman data seleksi ke endpoint palsu        | Restriksi eksplisit domain tujuan pada direktif CSP connect-src.                                          |
| <b>UI Redressing / Clickjacking</b> | Menengah          | Penanaman aplikasi dalam iframe tersembunyi      | Pemasangan skrip <i>frame-busting</i> pada level JavaScript <i>runtime</i> untuk melengkapi CSP meta tag. |

## Spesifikasi Fungsional dan Desain Antarmuka Pengguna

Pengalaman visual dan interaksi aplikasi dirancang mengikuti standar industri photo proofing profesional seperti Pixieset, dengan mengutamakan performa rendering berkecepatan tinggi, kejelasan status kuota, dan kemudahan pengoperasian tanpa kurva belajar bagi klien. Pondasi desain menggunakan ekosistem @shadcn/ui, memadukan komponen primitif Radix UI yang dapat diakses (*accessible*) dengan utilitas Tailwind CSS untuk memastikan kerapian tata letak dan estetika modern berstandar enterprise.

### Alur Perjalanan Pengguna (User Journey Flow)

Siklus peninjauan foto oleh klien melewati tahapan interaksi terstruktur:

1. **Akses Galeri dan Inisialisasi PWA:** Klien membuka tautan galeri statis yang dibagikan oleh fotografer atau meluncurkan aplikasi yang telah diinstal ke layar utama perangkat seluler melalui prompt PWA.
2. **Sinkronisasi Katalog Foto:** Aplikasi secara otomatis memvalidasi pengenalan folder Google Drive dari URL dan memuat daftar berkas menggunakan Google Drive API v3 secara asinkron.
3. **Eksplorasi Galeri Masonry:** Foto disajikan dalam kisi adaptif multi-kolom yang mempertahankan rasio aspek asli gambar tanpa terpotong (*cropping*), dilengkapi dengan pemuatan bertahap (*lazy loading*) untuk menjaga kelancaran animasi.
4. **Kurasi dan Manajemen Kuota:** Klien menandai foto pilihan dengan menekan ikon centang atau hati pada kartu foto. Panel kendali mengambang (*floating dock*) di bagian bawah layar secara otomatis memperbarui indikator kuota secara real-time.
5. **Inspeksi Detail Resolusi Tinggi:** Klien dapat membuka mode peninjauan layar penuh

(*lightbox*) untuk memeriksa ketajaman detail foto, melakukan perbesaran (*zoom*), berpindah antar foto dengan gestur sentuh atau tombol panah keyboard, serta menambahkan catatan khusus revisi.

6. **Kompilasi dan Pengiriman Hasil:** Setelah jumlah foto yang dipilih sesuai dengan kuota paket layanan, sistem mengunci opsi penambahan lebih lanjut dan membuka dialog konfirmasi untuk mengirimkan hasil kurasi kembali kepada fotografer.

## Struktur Data dan Manajemen State

Pengelolaan status seleksi foto diatur di sisi klien menggunakan Zustand yang dikombinasikan dengan pustaka `idb-keyval` untuk persistensi otomatis ke IndexedDB. Arsitektur ini menjamin bahwa seluruh foto yang telah ditandai oleh klien tidak akan terhapus meskipun halaman web dimuat ulang secara tidak sengaja atau peramban ditutup.

```
export interface PhotoMetadata {
  id: string;
  name: string;
  thumbnailUrl: string;
  previewUrl: string;
  width?: number;
  height?: number;
  sizeBytes?: string;
}

export interface ClientSelectionSession {
  projectId: string;
  clientName: string;
  clientContact: string;
  maxQuota: number;
  selectedPhotoIds: string[];
  revisioNotes: Record<string, string>;
  isLocked: boolean;
  lastModified: number;
}
```

## Mekanisme Sinkronisasi Status dan Ekspor Data Tanpa Server

Ketiadaan basis data terpusat pada arsitektur GitHub Pages diatasi melalui penyediaan tiga saluran sinkronisasi data yang terpisah secara independen. Pendekatan ini memberikan fleksibilitas bagi fotografer untuk memilih metode transmisi data yang paling sesuai dengan kebutuhan alur kerja mereka.

### Saluran Webhook Ingestion Google Apps Script

Fotografer dapat memanfaatkan Google Apps Script (GAS) yang dihubungkan langsung ke lembar kerja Google Sheets sebagai penyimpan data formulir. Script disebar (deploy)

sebagai Web App dengan izin eksekusi sebagai pemilik dokumen dan akses terbuka untuk siapa saja.

Kendala teknis mendasar pada arsitektur ini adalah pemblokiran CORS saat peramban mengirimkan permintaan POST berformat JSON ke endpoint Google Script. Masalah ini diatasi secara elegan dengan mengirimkan muatan data menggunakan header Content-Type: text/plain;charset=utf-8. Format *simple request* ini tidak memicu pemeriksaan *preflight* OPTIONS peramban, sementara fungsi doPost(e) di sisi Google Apps Script tetap dapat mengurai data melalui JSON.parse(e.postData.contents) secara transparan:

```
// Google Apps Script (Code.gs)
function doPost(e) {
  try {
    const data = JSON.parse(e.postData.contents);
    const sheet =
      SpreadsheetApp.getActiveSpreadsheet().getActiveSheet();

    sheet.appendRow([
      new Date(),
      data.clientName,
      data.clientContact,
      data.projectId,
      data.selectedFiles.length,
      data.selectedFiles.map(function(item) { return item.name;
    }).join(", "),
      JSON.stringify(data.revisionNotes)
    ]);

    return ContentService.createTextOutput(
      JSON.stringify({ status: "success", count:
data.selectedFiles.length })
    ).setMimeType(ContentService.MimeType.JSON);
  } catch (err) {
    return ContentService.createTextOutput(
      JSON.stringify({ status: "error", message: err.toString() })
    ).setMimeType(ContentService.MimeType.JSON);
  }
}
```

Di sisi aplikasi klien React, transmisi dieksekusi dengan mengaktifkan parameter redirect: "follow" untuk menangani mekanisme pengalihan HTTP 302 yang secara alami diterapkan oleh infrastruktur Google Web App:

```
export async function transmitSelectionData(endpointUrl: string,
payload: object): Promise<boolean> {
  try {
    const response = await fetch(endpointUrl, {
      method: "POST",
      headers: {
        "Content-Type": "text/plain;charset=utf-8",
      },
    },
```

```

        body: JSON.stringify(payload),
        redirect: "follow",
    });
    const result = await response.json();
    return result.status === "success";
} catch (error) {
    console.error("Transmission error:", error);
    return false;
}
}

```

## Saluran Sinkronisasi Status Berbasis URL Hash Kompresi

Sebagai alternatif tanpa integrasi Google Apps Script, sistem mendukung arsitektur sinkronisasi *stateless*. Seluruh larik (*array*) pengenalan berkas yang dipilih dikompresi menggunakan algoritma Lempel-Ziv-Welch via pustaka lz-string dan dimasukkan ke dalam fragmen hash URL

(#proof=...):

```

import { compressToEncodedURIComponent,
decompressFromEncodedURIComponent } from "lz-string";

export function generateCompressedProofUrl(selectedIds: string[]):
string {
    const serialized = JSON.stringify(selectedIds);
    const compressed = compressToEncodedURIComponent(serialized);
    const baseUrl = window.location.origin + window.location.pathname;
    return `${baseUrl}#proof=${compressed}`;
}

export function extractSelectionFromUrl(): string[] {
    const hash = window.location.hash;
    if (!hash.startsWith("#proof=")) return [];
    try {
        const compressed = hash.replace("#proof=", "");
        const decompressed =
decompressFromEncodedURIComponent(compressed);
        return decompressed ? JSON.parse(decompressed) : [];
    } catch {
        return [];
    }
}

```

Fragmen URL yang berada setelah tanda pagar (#) tidak pernah dikirimkan ke server web statis GitHub Pages dalam transaksi HTTP, sehingga privasi data seleksi klien tetap terjaga secara mutlak. Tautan yang dihasilkan dapat langsung dibagikan oleh klien ke nomor WhatsApp fotografer secara terotomasi via URL skema <https://wa.me/?text=....>

## Format Ekspor Pasca-Produksi

Untuk mempercepat alur kerja di studio pasca-seleksi, aplikasi menyediakan dua format ekspor langsung:

1. **String Filter Adobe Lightroom / Capture One:** Tombol salin satu-klik yang memformat nama-nama file terpilih dengan pemisah koma atau spasi standar (contoh: IMG\_1024.JPG, IMG\_1028.JPG, IMG\_1050.JPG). Fotografer dapat langsung menempelkan string ini ke bilah pencarian pustaka Lightroom untuk mengisolasi file terpilih secara instan.
2. **Manifest Terstruktur (CSV / JSON):** File unduhan berformat tabel yang memuat nama berkas, ukuran, ID Google Drive, stempel waktu seleksi, serta instruksi khusus penyuntingan dari klien.

## Konfigurasi Progressive Web App (PWA) dan Strategi Penyimpanan Luring

Integrasi PWA memastikan klien dapat memasang galeri foto ke dalam sistem operasi perangkat seluler mereka tanpa melalui toko aplikasi formal, menghadirkan nuansa aplikasi native tanpa bilah peramban, serta mempertahankan operasionalitas saat koneksi data terputus.

Otomatisasi PWA dikonfigurasi melalui vite-plugin-pwa yang terhubung dengan pustaka Workbox. Peta penyimpanan *cache* dipisahkan menjadi dua kompartemen: berkas statis inti aplikasi (*app shell*) dan berkas gambar eksternal yang di-host pada CDN Google.

```
// vite.config.ts
import { defineConfig } from "vite";
import react from "@vitejs/plugin-react";
import { VitePWA } from "vite-plugin-pwa";

export default defineConfig({
  plugins: [
    react(),
    VitePWA({
      registerType: "autoUpdate",
      includeAssets: ["favicon.ico", "apple-touch-icon.png",
"masked-icon.svg"],
      manifest: {
        name: "Lumina Proof Studio",
        short_name: "LuminaProof",
        description: "Platform Seleksi dan Kurasi Foto Profesional",
        theme_color: "#0a0a0a",
        background_color: "#0a0a0a",
        display: "standalone",
        orientation: "portrait",
        start_url: "./",
        scope: "./",
        icons: [
          {

```

```

        src: "pwa-192x192.png",
        sizes: "192x192",
        type: "image/png",
      },
      {
        src: "pwa-512x512.png",
        sizes: "512x512",
        type: "image/png",
        purpose: "any maskable",
      },
    ],
  },
  workbox: {
    globPatterns: ["**/*.{js,css,html,svg,png,woff2}"],
    runtimeCaching: [
      {
        urlPattern:
/^https:\\/\\/(lh3|lh4|lh5|lh6)\\.googleusercontent\\.com\\/\\.*/i,
        handler: "StaleWhileRevalidate",
        options: {
          cacheName: "google-thumbnails-cache",
          expiration: {
            maxEntries: 500,
            maxAgeSeconds: 60 * 60 * 24 * 7,
            purgeOnQuotaError: true,
          },
          cacheableResponse: {
            statuses: [0, 200],
          },
        },
      },
      {
        urlPattern:
/^https:\\/\\/www\\.googleapis\\.com\\/drive\\/v3\\/files\\.*/i,
        handler: "NetworkFirst",
        options: {
          cacheName: "google-api-metadata-cache",
          networkTimeoutSeconds: 4,
          expiration: {
            maxEntries: 30,
            maxAgeSeconds: 60 * 60 * 24,
          },
        },
      },
    ],
  },
},
),
],

```

```

    base: "./",
  });

```

| Entitas Data                       | Media Penyimpanan              | Strategi Caching                              | Perilaku Operasional                                                           |
|------------------------------------|--------------------------------|-----------------------------------------------|--------------------------------------------------------------------------------|
| <b>App Shell (JS, CSS, HTML)</b>   | Peramban<br>CacheStorage       | <i>Cache-First</i> via<br>Workbox Precaching  | Aplikasi dapat<br>diluncurkan secara<br>instan tanpa koneksi<br>jaringan.      |
| <b>Thumbnail Foto<br/>Google</b>   | Peramban<br>CacheStorage       | <i>Stale-While-Revalidate</i>                 | Menampilkan versi<br>lokal tersimpan;<br>memperbarui asinkron<br>saat online.  |
| <b>Metadata File<br/>Direktori</b> | IndexedDB<br>(photo_catalog)   | <i>Network-First</i> dengan<br>Fallback Cache | Struktur album dan<br>nama berkas tetap<br>dapat ditinjau secara<br>luring.    |
| <b>Status Seleksi Klien</b>        | IndexedDB<br>(selection_state) | Persistensi Lokal<br>Zustand Real-time        | Klien bebas menandai<br>foto luring; status<br>antrean dikirim saat<br>online. |

## Otomatisasi CI/CD dan Penerapan Berkelanjutan via GitHub Actions

Aplikasi dikonfigurasi untuk menjalankan otomatisasi pengujian, verifikasi tipe data, audit keamanan dependensi, kompilasi aset statis, dan penerapan ke GitHub Pages secara atomik menggunakan GitHub Actions.

### Konfigurasi Alur Kerja Penerapan (deploy.yml)

Implementasi alur kerja memanfaatkan tindakan resmi actions/deploy-pages@v4 yang terintegrasi dengan penyedia identitas OpenID Connect (OIDC) GitHub Pages, menerapkan prinsip pembatasan hak akses token minimal:

```
name: Continuous Integration & Deployment Pipeline
```

```
on:
```

```
  push:
```

```
    branches:
```

```
      - main
```

```
  workflow_dispatch:
```

```
permissions:
```

```
  contents: read
```

```
  pages: write
```

```
  id-token: write
```

```

concurrency:
  group: "pages"
  cancel-in-progress: true

jobs:
  validate-and-audit:
    name: Code Quality & Security Gate
    runs-on: ubuntu-latest
    steps:
      - name: Checkout Codebase
        uses:
actions/checkout@v4 [span_126] (start_span) [span_126] (end_span)

      - name: Setup Node.js Environment
        uses:
actions/setup-node@v4 [span_127] (start_span) [span_127] (end_span)
        with:
          node-version: 20
          cache: "npm"

      - name: Install Dependencies
        run: npm ci

      - name: Run ESLint Linter

[span_114] (start_span) [span_114] (end_span) [span_119] (start_span) [span_
119] (end_span)      run: npm run lint

      - name: Typecheck TypeScript Codebase
        run: npx tsc --noEmit

      - name: Audit Production Dependencies
        run: npm audit --audit-level=high

  build-and-deploy:
    name: Build Static Distribution & Deploy
    needs: validate-and-audit
    runs-on: ubuntu-latest
    environment:
      name: github-pages
      url: ${ steps.deployment.outputs.page_url }
    steps:
      - name: Checkout Codebase
        uses:
actions/checkout@v4 [span_128] (start_span) [span_128] (end_span)

      - name: Setup Node.js Environment
        uses:

```

```

actions/setup-node@v4 [span_129] (start_span) [span_129] (end_span)
  with:
    node-version: 20
    cache: "npm"

- name: Install Dependencies
  run: npm ci

- name: Build Static Web App
  env:

[span_115] (start_span) [span_115] (end_span) [span_120] (start_span) [span_
120] (end_span) VITE_GOOGLE_API_KEY: ${ secrets.PROD_GOOGLE_API_KEY
}} [span_134] (start_span) [span_134] (end_span)
  run: npm run build

- name: SPA Route Fallback & Jekyll Bypass
  run: |

[span_116] (start_span) [span_116] (end_span) [span_121] (start_span) [span_
121] (end_span) cp dist/index.html
dist/404.html [span_135] (start_span) [span_135] (end_span) [span_137] (star
t_span) [span_137] (end_span)
  touch dist/.nojekyll

- name: Configure GitHub Pages
  uses: actions/configure-pages@v5

- name: Upload Pa[span_130] (start_span) [span_130] (end_span)ges
Artifact
  uses: actions/upload-pages-artifact@v3
  with:
    [span_131] (start_span) [span_131] (end_span)path: "./dist"

- name: Deploy to GitHub Pages Engine
  id: deployment
  uses:
actions/deploy-pages@v4 [span_139] (start_span) [span_139] (end_span) [span
_140] (start_span) [span_140] (end_span)

```

## Mitigasi Khusus Perutean Single Page Application (SPA)

Server statis GitHub Pages tidak memiliki modul perutean dinamis internal seperti berkas konfigurasi Nginx `try_files`[span\_51](start\_span)[span\_51](end\_span). Akibatnya, pemuatan ulang peramban pada rute sekunder (contoh: <https://username.github.io/gallery/album-wedding>) akan memunculkan respons 404 Not Found.

Dua langkah penting diintegrasikan ke dalam langkah pembangunan pipeline:

1. **Penyalinan Dokumen 404.html:** Perintah `cp dist/index.html dist/404.html` menduplikasi berkas HTML utama ke berkas kesalahan default. Ketika rute dalam diakses, GitHub Pages menyajikan dokumen ini dengan kode status 404, memungkinkan perute sisi klien (React Router) mengambil alih path URL dan merender komponen yang tepat secara transparan.
2. **Penyertaan Berkas Penanda .nojekyll:** Eksekusi `touch dist/.nojekyll` menonaktifkan pemrosesan pra-kompilasi mesin Ruby Jekyll bawaan GitHub Pages. Hal ini menjamin bahwa seluruh berkas dan folder terkompilasi yang berawalan dengan garis bawah (seperti `_assets` pada hasil bundel Vite tertentu) tidak diabaikan oleh server.

## Matriks Kebutuhan Sistem dan Panduan Implementasi

Spesifikasi fungsional dan teknis aplikasi dirangkum dalam matriks terstruktur sebagai acuan pemenuhan standar rekayasa perangkat lunak:

| Kode ID | Klasifikasi Persyaratan | Deskripsi Teknis Kebutuhan                                      | Derajat Kepentingan | Kriteria Penerimaan ( <i>Acceptance Criteria</i> )                          |
|---------|-------------------------|-----------------------------------------------------------------|---------------------|-----------------------------------------------------------------------------|
| REQ-F01 | Fungsional              | Pengambilan metadata direktori Google Drive via API v3.         | Kritis (P0)         | Mampu memuat 500 metadata foto dalam waktu < 2.5 detik pada jaringan 4G.    |
| REQ-F02 | Fungsional              | Rendering galeri kisi masonry adaptif responsif.                | Kritis (P0)         | Bebas layout shift (CLS < 0.05); scrolling konstan 60 FPS pada mobile.      |
| REQ-F03 | Fungsional              | Penandaan seleksi foto dan pembatasan kuota paket.              | Kritis (P0)         | Sistem mengunci tombol seleksi lanjutan ketika kuota tepat terpenuhi.       |
| REQ-F04 | Fungsional              | Antarmuka Lightbox layar penuh dengan navigasi keyboard/gestur. | Tinggi (P1)         | Latensi respons transisi antar gambar di bawah 100 ms.                      |
| REQ-F05 | Fungsional              | Ekspor daftar foto terpilih ke format teks Adobe Lightroom.     | Tinggi (P1)         | Menghasilkan string delimitasi koma yang valid disalin ke clipboard sistem. |
| REQ-F06 | Fungsional              | Transmisi formulir otomatis ke Google Sheets via GAS.           | Sedang (P2)         | Transmisi data sukses tanpa pemblokiran kebijakan CORS                      |

| Kode ID        | Klasifikasi Persyaratan | Deskripsi Teknis Kebutuhan                                        | Derajat Kepentingan | Kriteria Penerimaan ( <i>Acceptance Criteria</i> )                       |
|----------------|-------------------------|-------------------------------------------------------------------|---------------------|--------------------------------------------------------------------------|
|                |                         |                                                                   |                     | peramban.                                                                |
| <b>REQ-N01</b> | Non-Fungsional          | Kepatuhan spesifikasi PWA dan fungsionalitas luring.              | Kritis (P0)         | Skor Lighthouse PWA 100/100; validasi status installable di Android/iOS. |
| <b>REQ-N02</b> | Non-Fungsional          | Perlindungan aplikasi melalui Content Security Policy deklaratif. | Kritis (P0)         | Tidak ada pelanggaran kebijakan keamanan skrip pada console browser.     |
| <b>REQ-N03</b> | Non-Fungsional          | Infrastruktur terdistribusi statis tanpa server konvensional.     | Kritis (P0)         | Aplikasi beroperasi 100% pada GitHub Pages dengan biaya server \$0.      |
| <b>REQ-N04</b> | Non-Fungsional          | Otomatisasi pipeline pengujian dan penerapan terkelola.           | Tinggi (P1)         | Seluruh alur kerja CI/CD GitHub Actions selesai dalam waktu < 3 menit.   |

Tahapan eksekusi teknis untuk membangun sistem dari tahap inisialisasi hingga peluncuran produksi dilakukan melalui langkah-langkah terstruktur berikut:

Proses rekayasa dimulai dengan pembentukan proyek baru menggunakan bundler Vite dengan template React dan TypeScript. Setelah struktur proyek terbentuk, seluruh dependensi utilitas seperti Tailwind CSS, primitif Radix UI, ikon Lucide, pustaka manajemen status Zustand, modul IndexedDB, algoritma kompresi data lz-string, serta plugin vite-plugin-pwa dipasang ke dalam lingkungan pengembangan lokal:

```
# Inisialisasi basis kode Vite React TypeScript
npm create vite@latest lumina-proofing -- --template react-ts
cd lumina-proofing
```

```
# Konfigurasi Tailwind CSS dan plugin utilitas
npm install -D tailwindcss postcss autoprefixer
npx tailwindcss init -p
```

```
# Pemasangan dependensi antarmuka, state engine, dan kapabilitas PWA
npm install @radix-ui/react-dialog @radix-ui/react-slot clsx
tailwind-merge lucide-react
npm install zustand idb-keyval lz-string
npm install -D vite-plugin-pwa
```

Konfigurasi path alias `@/*` diterapkan pada berkas `tsconfig.json` dan `vite.config.ts` untuk

memfasilitasi integrasi komponen modular bergaya shadcn/ui secara konsisten.

Tahap kedua berfokus pada penyediaan kredensial pada Google Cloud Console. Pengembang membuat proyek baru, mengaktifkan Google Drive API melalui pustaka API, dan menerbitkan satu API Key khusus. Kunci ini dikunci secara ketat dengan menetapkan pembatasan API tunggal ke Google Drive API serta memasang HTTP Referrer Restrictions ke alamat GitHub Pages produksi (<https://<username>.github.io/>\*) dan alamat lokal peramban.

Tahap ketiga melibatkan penyusunan backend serverless berbasis Google Apps Script jika fitur transmisi otomatis ke spreadsheet diinginkan. Lembar kerja Google Sheets dibuat khusus sebagai basis data, diikuti pembukaan editor script bawaan untuk menanamkan fungsi pengurai doPost(e). Script kemudian dipublikasikan sebagai Web App dengan izin eksekusi sebagai pemilik akun dan visibilitas publik, menghasilkan URL endpoint unik yang akan dipanggil oleh fungsi fetch aplikasi.

Tahap penutup mengintegrasikan pipeline CI/CD dengan mendaftarkan API Key Google ke dalam fitur rahasia repositori GitHub melalui menu Settings > Secrets and variables > Actions dengan nama PROD\_GOOGLE\_API\_KEY. Berkas konfigurasi alur kerja `.github/workflows/deploy.yml` dimasukkan ke dalam repositori, dan pengaturan deployment GitHub Pages dialihkan ke opsi GitHub Actions. Setelah seluruh kode didorong (*push*) ke cabang utama main, pipeline Actions akan memverifikasi integritas kode dan meluncurkan platform proofing ke CDN publik secara instan.

## Kesimpulan dan Arah Rekayasa Lanjutan

Rancangan sistem yang dipaparkan dalam dokumen ini membuktikan bahwa platform seleksi foto mandiri berstandar industri dapat dibangun tanpa ketergantungan pada server komputasi backend berbayar. Integrasi antara Google Drive API v3 dan hosting statis GitHub Pages berhasil meniadakan biaya infrastruktur rutin sekaligus memanfaatkan ekosistem penyimpanan Google Workspace yang telah dimiliki fotografer.

Kekuatan arsitektur ini bertumpu pada tiga pilar utama:

1. **Keandalan Akses Konten:** Resolusi visual langsung melalui CDN `lh3.googleusercontent.com` mengatasi perubahan pemblokiran sematan lama Google, memberikan kecepatan rendering yang konsisten pada resolusi adaptif.
2. **Pertahanan Siber Statis:** Isolasi API Key dengan restriksi HTTP Referrer yang dipadukan dengan kebijakan Content Security Policy deklaratif memastikan bahwa ketiadaan server backend tidak mengurangi tingkat keamanan aplikasi.
3. **Fleksibilitas Portabilitas Data:** Kehadiran opsi transmisi ganda, baik melalui webhook Google Apps Script yang terbebas dari hambatan CORS maupun melalui kompresi status berbasis URL hash lz-string, memberikan kedaulatan data penuh bagi pengguna.

Penerapan standar antarmuka modern berbasis shadcn/ui dan fungsionalitas Progressive Web App (PWA) menjamin kenyamanan interaksi klien setara aplikasi komersial, menghadirkan efisiensi alur kerja pasca-produksi yang optimal bagi fotografer profesional.

## Karya yang dikutip

1. Deploying a Static Site - Vite, <https://vite.dev/guide/static-deploy>
2. Day 21 - Deploy the Github Profile Project to Github Pages, <https://dev.to/railsstudent/day-21-deploy-the-github-profile-project-to-github-pages-p39>
3. CRUD

Operations on Static File Sites,  
<https://douglasduhaime.com/posts/crud-operations-on-static-file-sites.html> 4. Photography Portfolio Template | Masonry Gallery - Lovable,  
<https://lovable.dev/templates/websites/portfolio/masonry-photographer> 5. Shadcn Gallery Blocks - 52+ Sections - Shadcnblocks.com, <https://www.shadcnblocks.com/blocks/gallery> 6. shadcn/ui - The Foundation for your Design System, <https://ui.shadcn.com/> 7. Guide for using Content-Security-Policy HTTP header for ... - GitHub,  
<https://github.com/brycejech/content-security-policy> 8. Deploying a Static Vite App on GitHub Pages with GitHub Actions,  
<https://ucilasmana.medium.com/deploying-a-static-vite-app-on-github-pages-with-github-actions-03a67bb9ac4e> 9. vite-plugin-offline - npm.io, <https://npm.io/package/vite-plugin-offline> 10. Images from Google drive in embedded HTML no longer working,  
<https://support.google.com/sites/thread/253003338/images-from-google-drive-in-embedded-html-no-longer-working?hl=en> 11. Having trouble displaying an image from Google Drive,  
<https://stackoverflow.com/questions/77803187/having-trouble-displaying-an-image-from-google-drive> 12. Displaying files (e.g. images) stored in Google Drive on a website,  
<https://stackoverflow.com/questions/10311092/displaying-files-e-g-images-stored-in-google-drive-on-a-website> 13. Recursively List and Bulk Download Public Google Drive Folders via,  
<https://scriptroad.com/2026/recursively-list-and-bulk-download-public-google-drive-folders-via-api/> 14. Guide: How to add image to Google Drive™ and use it in signature?,  
<https://signaturky.com/kb/guide-how-to-add-image-to-google-drive-and-use-it-in-signature/> 15. How to get list of files by folder on Google Drive API - Stack Overflow,  
<https://stackoverflow.com/questions/24720075/how-to-get-list-of-files-by-folder-on-google-drive-api> 16. How to embed any image from google drive into your website with,  
<https://data-addict.jadynekena.com/how-to-embed-any-image-from-google-drive-into-your-website-with-this-always-working-solution/> 17. html - How do I display images from Google Drive on a website?,  
<https://stackoverflow.com/questions/15557392/how-do-i-display-images-from-google-drive-on-a-website> 18. Browser CORS issue when accessing Google Drive API using,  
<https://community.latenode.com/t/browser-cors-issue-when-accessing-google-drive-api-using-google-identity-authentication/29948> 19. Google Drive Connector | Prismatic Docs,  
<https://prismatic.io/docs/components/google-drive/> 20. Content Security Policy setting · community · Discussion #49832, <https://github.com/orgs/community/discussions/49832> 21. How to set Content Security Policy with a Meta Tag? - DEV Community,  
<https://dev.to/jacobandrewsky/how-to-set-content-security-policy-with-a-meta-tag-5gfm> 22. Add a Content Security Policy to a GitHub Pages Site - Isaac Smith,  
<https://www.isaacsmith.us/blog/2022/add-csp-to-github-pages> 23. Content Security Policy Cheat Sheet - GitHub,  
[https://github.com/OWASP/CheatSheetSeries/blob/master/cheatsheets/Content\\_Security\\_Policy\\_Cheat\\_Sheet.md](https://github.com/OWASP/CheatSheetSeries/blob/master/cheatsheets/Content_Security_Policy_Cheat_Sheet.md) 24. CRUD Using Google Apps Script - GitHub Gist,  
<https://gist.github.com/ex-preman/e892f72404a0e4779999f8b439cec1b1> 25. natalieandjordi/README.md at main · joadoumie ... - GitHub,  
<https://github.com/joadoumie/natalieandjordi/blob/main/README.md> 26. Getting CORS using JS and Google Apps script - Stack Overflow,  
<https://stackoverflow.com/questions/66234681/getting-cors-using-js-and-google-apps-script> 27. Google App Script Web App GET and POST request blocked by,  
<https://stackoverflow.com/questions/56502086/google-app-script-web-app-get-and-post-request-blocked-by-cors-policy> 28. Struggling with CORS in Google Apps Script? Here's the Fix -

Medium,

<https://diyavijay.medium.com/struggling-with-cors-in-google-apps-script-heres-the-fix-e3eec09f07dd>

29. Contact Form to Google Sheets — Without Writing Apps Script,

<https://rgforms.com/google-sheets-contact-form/> 30. Fixing CORS Errors in Google Apps Script | Lambda IITH, <https://iith.dev/blog/app-script-cors/> 31. Effective Strategies for Handling Long

Query Strings - Medium,

<https://medium.com/suyeonme/effective-strategies-for-handling-long-query-strings-b790e1fddd65> 32. Run in the cloud with molab - Marimo docs, <https://docs.marimo.io/guides/molab/> 33. How

to compress URL parameters - javascript - Stack Overflow,

<https://stackoverflow.com/questions/21802866/how-to-compress-url-parameters> 34. JWT

Decoder & Verifier - onDevTools, <https://www.ondetools.com/en/jwt-decoder> 35. GeoLibre

Architecture, <https://geolibre.app/architecture/> 36. OpenConstructionERP/CHANGELOG.md at main - GitHub,

<https://github.com/datadrivenconstruction/openconstructionerp/blob/main/CHANGELOG.md> 37.

How to Deploy Your React Application with Vite to GitHub Pages,

<https://dev.to/whoisclabs/github-actions-how-to-deploy-your-react-application-with-vite-to-github-pages-2cb8> 38. Deploying a Vite React TypeScript app to Github Pages using,

<https://levelup.gitconnected.com/deploying-a-vite-react-typescript-app-to-github-pages-using-git-hub-actions-jest-and-pnpm-as-a-a3461ef9c4ad> 39. GitHub actions environment variables are not loaded in Vite build,

<https://stackoverflow.com/questions/78649695/github-actions-environment-variables-are-not-loaded-in-vite-build> 40. Deploying a Vite App to GitHub Pages using GitHub Actions,

<https://javascript.plainenglish.io/deploying-a-vite-app-to-github-pages-using-github-actions-a-step-by-step-guide-ff63d93e4da2>